01 / ROLE AND ROADMAP

Neelanshu's Phase 1 work guide

PROJECT 11 / AUTOMATIC CLOTHING CATEGORY RECOGNITION

Inventory Image Tagging via Handcrafted Features and a Small CNN
Phase 1 focus: classical machine learning on Fashion-MNIST.

Your role: Evaluation and error-analysis lead

Define the metrics and prediction-file contract before results arrive.

The five-person division

Member	Primary ownership
Nakul	Integration, literature and report Repository, master notebook, report assembly
Sudeshhna	Data quality and EDA Loader, fixed split, audit, EDA figures
Puneet	Features, Logistic Regression and kNN Pixel/HOG features and four model rows
Samarth	Random Forest, SVM and run harness Four model rows and experiment logs
Neelanshu	Evaluation and error analysis Metrics, timing, complexity, error figures

How to use this guide

Read pages 2-4 first, then work through your practical pages in order. Pages 13-16 explain handoffs, viva preparation, the proposed schedule and sources. Your guide is a work plan and learning manual, not a completed experiment or a submission-ready report.

All five members must understand the entire pipeline. Ownership means you implement and explain an area; it does not mean the others can skip learning it. Each member also writes their report section, prepares their slides, and reviews another member's work.

Prepared 27 October 2026. All workload estimates, code organization, hyperparameter grids and internal milestones are recommendations, not faculty rules. No model scores have been invented.

02 / SHARED FOUNDATIONS

What Phase 1 actually requires

The supplied CSAI instructions are the course requirements. Your project description supplies the technical problem. This guide adds a practical way to satisfy both; it does not turn its suggestions into extra faculty requirements.

Course requirement	Evidence the team should prepare
Literature review	A cited comparison of relevant sources and the gap your experiments investigate.
Dataset quality and EDA	Shape/range/label checks, class counts, sample images and a written audit.
Feature engineering	Normalized flattened pixels and a justified HOG configuration.
Theoretical rigor	Explain each model, its assumptions, tuning choices and limitations.
Model application	Two representations with Logistic Regression, kNN, Random Forest and SVM.
GitHub and code quality	Reproducible code, clear README, saved settings, outputs and contribution evidence.
Presentation and individual viva	Every member presents their work and explains the shared experiment.
Project report	A coherent report in LaTeX/IEEE format, using the course template if supplied.

Scope boundary

Complete the classical ML component now. Save the CNN, pooling, dropout, CNN early stopping, AML-versus-CNN experiments and any implemented hybrid approach for Phase 2. A brief future-work paragraph is enough at this stage. The Phase 1 comparison is pixels versus HOG across four classical classifiers.

The attachment gives no Phase 1 evaluation date, page limit, presentation duration, grading weights, minimum accuracy, or required number of papers. Its September dates concern project/team selection. Check the course channel for later evaluation announcements; do not treat the proposed schedule here as an official deadline.

03 / SHARED FOUNDATIONS

Machine learning from zero

Here, **AML** means the classical/applied machine-learning part described in your project. You do not hand-write a rule saying "sleeves mean shirt." Instead, you show a model labeled examples so it can learn a rule from image measurements.

Term	Plain meaning in this project
Sample; label	One clothing image; its correct category number. The label is the answer.
Feature; representation	A numeric input. One image becomes either 784 pixel values or a HOG vector.
X and y	X is the feature matrix: one row per image. y is the matching list of labels.
Model; fit; predict	A learned decision rule; learn from X and y; assign labels to new X.
Parameter	Something learned during fitting, such as a Logistic Regression coefficient.
Hyperparameter	A setting chosen before fitting, such as k in kNN or C in an SVM.
Training / validation / test	Practice examples / development exam / final independent exam.
Overfitting	Doing very well on training data but failing to generalize to held-out data.
EDA	Exploratory data analysis: inspect the data before deciding how to model it.
Reproducibility	Another person can repeat the process using the same data, code and settings.

Walk through a single image

A grayscale image is a small grid of brightness values. Divide its raw pixel values by 255 to put them on a 0-1 scale. Flatten the grid to a vector, or compute a HOG description of local edge directions. A fitted classifier then maps that vector to one of the category IDs. Map the ID to the correct name for display.

Your first learning exercise

Before your assigned work, use a notebook to print an array's shape, show one image, print its label, and call fit then predict on a tiny example. Explain aloud why a row of X must stay attached to the same row of y. If that connection is broken, model training becomes meaningless.

Important nuance: flattening does not erase pixel values or their recoverable ordering. It gives a conventional classifier no built-in local-neighborhood or translation structure. HOG encodes local geometry, but its feature rule is fixed before classifier training.

04 / SHARED FOUNDATIONS

One shared, fair experiment

Freeze this protocol before comparing models

- Keep the official training and test partitions separate. Create a stratified 80/20 split of the official training partition with random_state=42: 48,000 fit images and 12,000 validation images. Save the indices once; everyone reuses them.
- Use two fixed representations and four model families: pixels/HOG × Logistic Regression/kNN/Random Forest/SVM. This produces eight main comparison rows. Add one DummyClassifier reference row outside those eight.
- Train on the fit partition, choose settings using validation macro F1, and use validation accuracy as a tie-breaker. If still tied, choose the simpler or faster option using validation measurements. Log the rule before seeing test scores.
- Fit learned scalers only on fit data. A fixed divide-by-255 transform and fixed per-image HOG extraction do not learn dataset statistics. Use a Pipeline for StandardScaler plus the classifier.
- After freezing one configuration per row and naming the validation-selected overall model, evaluate the eight frozen models on the official test partition. Do not choose new hyperparameters or a new overall winner from test results.

Training size and compute policy

Default: all eight final models use the same 48,000-image fit partition and the same 12,000-image validation partition. Keep those fitted models for final testing; do not silently refit some models on all 60,000 training images. This preserves the same split for Phase 2.

If compute makes this infeasible, agree before final runs on one saved stratified 12,000-image subset drawn only from the 48,000 fit images. Use that identical subset for all eight rows, retain the same validation/test partitions, and label the table "12,000 fit images per model." Exploratory subset runs must never masquerade as full-size results.

Required result package

Report accuracy, macro F1, recall for all ten classes, training time, inference time with a stated protocol, and a meaningful model-complexity description. Explain when a classical model has no neural-network-style parameter count. Save all eight rows, including weak runs; discuss shirt/T-shirt/pullover confusion using observed examples.

Protocol choices on this page are recommendations for consistency. The official split and the stated project metrics come from your project description; the specific validation fraction, seed and selection rule are proposed here.

05 / YOUR WORK

Your ownership and first session

You establish what counts as a good result and make sure the reported numbers actually measure it. Your role combines code, statistical care and visual explanation. You own the final evidence, but model owners remain responsible for their implementations and their interpretations.

What you personally build

- **src/evaluate.py:** shared metric calculation, prediction-schema checks, per-class recall, confusion matrices and complexity summaries.
- **notebooks/04_evaluation.ipynb:** dummy reference, validation audit, frozen test evaluation, timing benchmark and paired error analysis.
- **Results and figures:** eight-row metric table, ten-class recall table, count/normalized confusion matrices, representative error panels and reproducible timing records.
- **Written analysis:** explain model/feature trade-offs, the limits of the evidence, and why a CNN is a future hypothesis. Verify every number used in Nakul's report and slides.

First 90 minutes

0-20 min: calculate accuracy on ten toy predictions by hand. **20-45 min:** learn TP, FP, FN, precision, recall and F1 using a single category against the rest. **45-65 min:** generate a toy confusion matrix and read its rows. **65-90 min:** agree the prediction-file schema with Puneet and Samarth, then write a function that rejects misaligned rows.

Your dummy baseline

Fit DummyClassifier(strategy="most_frequent") on the same fit labels and evaluate it on validation. It ignores image content and gives a basic reference. Do not count it as one of the four required model families, or copy an expected score instead of running it.

Your handoff is accepted when

Nakul can regenerate every report result from saved predictions and metadata. Model owners understand the measured errors. A teammate can follow your timing protocol and obtain a sensible repeated measurement on the stated machine.

Suggested effort: 20-26 hours including metrics, final audit, runtime work, figures, report writing and rehearsal. You can implement most tools using toy predictions before final models arrive.

06 / YOUR WORK

Metrics: calculate before you code

Use the same metric definitions for every row. A high overall score can coexist with poor recall for a particular category. That is why the project asks for accuracy, macro F1 and per-class recall. [R10]

Accuracy

Accuracy = number correct / number evaluated. Toy example: 820 correct predictions out of 1,000 gives 0.82, or 82%. This is an arithmetic illustration, not a score for your models.

One category versus the rest

For "shirt," **TP** means actual shirts predicted as shirt. **FN** means actual shirts predicted as another category. **FP** means other garments predicted as shirt. **Precision = TP/(TP+FP)**. **Recall = TP/(TP+FN)**. **F1 = 2TP/(2TP+FP+FN)**.

Toy example: 100 actual shirts, 70 correctly predicted shirts, and 80 total shirt predictions. TP=70, FN=30 and FP=10. Recall is 70%, precision is 87.5%, and F1 is about 77.8%. Recall answers "how many real shirts did we recover?" Precision answers "how often was a shirt prediction correct?"

Macro F1

Compute each of the ten class F1 scores separately and average them: macro F1 = (F1_0 + ... + F1_9)/10. It is not the F1 calculated from averaged precision and recall. It gives each category equal weight even if supports differ. Use an explicit label list 0-9 and a stated policy for undefined precision/F1.

Per-class recall and support

Support is the number of actual examples in that category in the evaluated partition. Save all ten recalls with supports, not just the weakest class. If support is zero in a toy set, say so; do not interpret a forced zero as evidence of model failure.

Difference versus percentage points

If an illustrative score changes from 0.80 to 0.83, that is +0.03 on the 0-1 scale or +3 percentage points. It is a 3.75% relative increase. Use percentage points when comparing accuracy percentages; avoid ambiguous "3% better" wording.

Do not assume two balanced partitions make all metrics redundant. Even when supports are balanced, F1 includes false positives and can show different weaknesses from accuracy or recall alone.

07 / YOUR WORK

Implement metrics and alignment checks

```
import numpy as np
from sklearn.metrics import (accuracy_score, f1_score,
                             recall_score, confusion_matrix)

LABELS = np.arange(10)
def evaluate_labels(y_true, y_pred):
    y_true, y_pred = np.asarray(y_true), np.asarray(y_pred)
    assert y_true.ndim == y_pred.ndim == 1
    assert len(y_true) == len(y_pred) and len(y_true) > 0
    assert np.isin(y_true, LABELS).all()
    assert np.isin(y_pred, LABELS).all()
    cm = confusion_matrix(y_true, y_pred, labels=LABELS)
    support = cm.sum(axis=1)
    recalls = recall_score(y_true, y_pred, labels=LABELS,
                           average=None, zero_division=0)
    return {
        "accuracy": accuracy_score(y_true, y_pred),
        "macro_f1": f1_score(y_true, y_pred, labels=LABELS,
                             average="macro", zero_division=0),
        "recall": recalls, "support": support, "cm": cm}
```

A correct formula cannot repair incorrect rows

- Require exactly one prediction per expected ID, with the correct source_split and stage. Reject duplicate IDs, missing IDs, extra IDs or inconsistent run IDs.
- Join predictions to the authoritative label array using original IDs. Compare exported y_true to authoritative labels; do not trust the exported column without checking.
- Reorder rows by the agreed ID order before comparing two models. Equal-length arrays are not proof of alignment.
- Check confusion-matrix sum equals evaluated N, its diagonal sum divided by N equals accuracy, and each row sum matches that category's support.

Meaningful toy verification

Use a small hand-constructed label set with known mistakes and verify one recall and the accuracy by hand. Also confirm that an intentionally duplicated or missing ID is rejected. These checks test real failure modes. You do not need a large test suite that merely repeats the metric library's formulas.

Save outputs independently of notebook display

Write a metrics JSON, a ten-row recall CSV and a confusion-matrix CSV for each run/stage. Keep the raw prediction CSV. Figures are views of these saved results; screenshots alone cannot reproduce the analysis. Use consistent category order and explicit units in every table.

08 / YOUR WORK

Read and compare confusion matrices

Use rows = true categories and columns = predicted categories, with the same ten-class order in every plot. The diagonal counts correct predictions. Cell (6,0), for example, counts true shirts predicted as T-shirt/top. Reversing the axes reverses the meaning of the story.

Make two versions for selected frozen rows

Count matrix: displays raw numbers and makes error volume visible. **Row-normalized matrix:** divides each row by its support and shows where actual examples of a class go. The diagonal of this row-normalized matrix equals per-class recall. Do not call it a precision matrix.

```
cm = metrics["cm"]
support = cm.sum(axis=1, keepdims=True)
row_norm = np.divide(cm, support,
    out=np.zeros_like(cm, dtype=float), where=support != 0)

fig, ax = plt.subplots(figsize=(8, 7))
im = ax.imshow(row_norm, vmin=0, vmax=1, cmap="Blues")
ax.set_xticks(range(10), names, rotation=65, ha="right")
ax.set_yticks(range(10), names)
ax.set_xlabel("Predicted category")
ax.set_ylabel("True category")
ax.set_title("Row-normalized confusion | run ID | stage")
fig.colorbar(im, ax=ax, label="Fraction of true category")
fig.tight_layout()
fig.savefig("figures/confusion_row_normalized.png", dpi=200)
```

Replace the generic run/stage title and filename with actual identifiers. Annotate cells if legible, using readable font sizes and appropriate decimals. Keep the same 0-1 color scale when comparing row-normalized plots. Include the evaluated partition and training profile in the caption.

Choose comparisons that isolate a question

For the main representation story, compare pixels and HOG **within the same classifier**, using the frozen settings and same sample IDs. Comparing pixel LR to HOG SVM changes both representation and algorithm, so it cannot isolate the feature effect. You can still show the overall validation-selected model separately.

What to write

Identify the largest off-diagonal confusions by counts and row-normalized rates. Discuss shirt/T-shirt/top/pullover if they are actually prominent, but report other dominant errors if the data shows them. Never force the expected story onto contradictory results.

09 / YOUR WORK

Error examples and concise analysis

Error analysis connects metrics to actual images. It should explain what the models struggle with while acknowledging uncertainty. Clothing silhouettes can look similar at low resolution, but one selected image is not enough to prove why a model failed.

Use a reproducible selection rule

- On validation, define a deterministic policy: for each prominent confusion pair, choose up to five misclassified examples by a fixed seed or smallest original IDs. Include a few correct examples as context.
- For pixels versus HOG within a classifier, align both prediction arrays to the same IDs. Form four groups: both correct, only pixels correct, only HOG correct, both wrong.
- Display the same raw images with true label and both predictions. Save each original ID and the selection rule. Do not choose only examples where the preferred method looks impressive.
- After final freeze, test error panels are descriptive only. If they reveal a weakness, document it as a limitation/future hypothesis rather than tuning the tested system.

Useful per-example fields

source_split, sample_id, true category, predicted category, classifier, representation, stage, correct/incorrect, comparison-group membership and a brief visual observation. Do not display unsupported confidence percentages from an uncalibrated decision score.

Write three short analysis paragraphs

1. What happened: cite the observed confusion counts and class recalls. **2. What the examples suggest:** describe visible ambiguities such as shared outline or unclear sleeves, qualifying them as possible explanations. **3. What follows:** state whether HOG helped that confusion, what costs it introduced, and what remains unresolved.

Claims to avoid

"HOG understands sleeves" is too strong: it measures local gradient structure. "The labels are wrong" is too strong without independent verification. "The CNN will solve it" is untested. "Our model works for any online store" exceeds evidence from a small grayscale benchmark.

Phase 1 conclusion template

"On the fixed split and shared training size, [representation/model] achieved [measured metrics]. The main remaining errors involved [observed pairs]. [Feature choice] changed [specific recall/F1] at [runtime/storage cost]. The result motivates testing learned spatial features in Phase 2, while real catalogue performance remains unmeasured." Use only actual numbers and supported claims.

10 / YOUR WORK

Measure inference time fairly

Timing results depend on machine, software, thread count, batch size and what the timer includes. All final models should be benchmarked on one machine/session under a documented policy. Never compare one member's laptop time to another member's hosted GPU time as if it were a model property.

Report two boundaries

Pipeline prediction: start from cached unscaled features; include any learned scaler plus classifier.predict. **End to end:** start from in-memory raw uint8 images; include 0-1 conversion, flattening/HOG, learned scaler and classification. Exclude disk reads and model loading from both and say so. Report feature extraction separately if useful.

Proposed benchmark recipe

After freeze, use the same fixed 1,024 official-test image IDs for every row, processed in batches of 128. Warm up once, then perform five timed repetitions. Report median milliseconds per image as amortized batch cost, along with spread and hardware/software details. This is not single-request latency. Use validation images for development timing and tie-breaks before freeze.

```
import time
import numpy as np

def benchmark(predict_fn, inputs, batch_size=128, repeats=5):
    # predict_fn accepts one input batch; model already loaded.
    predict_fn(inputs[:min(batch_size, len(inputs))])
    seconds = []
    for _ in range(repeats):
        start = time.perf_counter()
        for j in range(0, len(inputs), batch_size):
            predict_fn(inputs[j:j + batch_size])
        seconds.append(time.perf_counter() - start)
    ms = 1000 * np.asarray(seconds) / len(inputs)
    return {"median_ms_per_image": float(np.median(ms)),
            "min_ms_per_image": float(ms.min()),
            "max_ms_per_image": float(ms.max())}
```

For Pipeline timing use predict_fn=pipe.predict and cached unscaled features. For end-to-end timing wrap pipe.predict(make_features(raw_batch, feature_name)), using the saved feature configuration. Ensure the wrapper recomputes features inside the timed region instead of reading the cache.

Record CPU, RAM, OS, Python/library versions, batch size, number of images, repeats, thread policy and whether other heavy tasks were running. Set thread environment variables before library imports if controlling native BLAS threads. Time classical models on CPU consistently; a GPU is irrelevant here.

11 / YOUR WORK

Complexity, storage and final test audit

Count what each model actually stores

Model	Recommended complexity evidence
Logistic Regression	coef_.size + intercept_.size for the classifier; list scaler statistics separately. State feature dimension.
kNN	No learned coefficient count. Report stored rows × feature dimension, approximate stored-feature bytes and serialized size.
Random Forest	Tree count, total node count and depth summary. Call these structural counts, not neural-equivalent parameters.
RBF SVC	Number of support vectors; support_vectors_.size + dual_coef_.size + intercept_.size as stored decision-array scalars. State that this is a storage proxy.
LinearSVC, if used	coef_.size + intercept_.size, with actual shapes. It is a different SVM implementation from kernel SVC.

For all models save the serialized Pipeline/bundle size in bytes and state what is included. File compression and metadata affect it; use the same serialization/compression policy. These counts describe computational/storage structure and are not directly interchangeable measures of capacity.

Before opening final test evaluation

- Require a frozen manifest naming eight final rows, their feature/model settings, data profile, artifact hashes and validation metrics. Verify the overall selected model was chosen using validation.
- Confirm every model used the same fit IDs and training size, all learned scalers were fitted on fit data, and prediction schemas pass alignment checks.
- Load the official test partition, compute features with the frozen rules, predict, and save every test prediction and metric. This step performs no fitting or hyperparameter selection.
- If a test result is surprising, inspect it for reporting. A genuine evaluation bug may require a documented correction and rerun; it does not justify test-guided model tuning.

Do not hide a test/validation disagreement

The validation-selected model may not have the largest test score among the eight frozen rows. Report both facts. Calling the largest observed test score a newly selected production winner would reuse the test set for selection. Describe it as a descriptive ranking and preserve the predeclared selection.

Repeated timing calls on frozen models are not new training trials. The important restriction is that test measurements do not guide changes to the evaluated model or its preprocessing.

12 / YOUR WORK

Your final evidence package

Files you deliver to Nakul

- **results/validation_summary.csv and test_summary.csv:** eight completed rows each, feature/model/settings identifiers, training size and accuracy/macro F1. The dummy reference is clearly marked separately.
- **results/per_class_recall.csv:** stage, run_id, label ID, category name, support and recall for all ten categories per run.
- **results/timing.csv and complexity.csv:** explicit units, boundary, machine/protocol metadata, repeated measurements, structural counts and storage bytes.
- **figures/:** readable confusion matrices and deterministic paired error panels with source CSVs and captions.
- **docs/evaluation_audit.md:** alignment, leakage, freeze and consistency checks; warnings; unresolved limits; and explanation of any corrected evaluation bug.

Results-table design

Keep the main accuracy/F1 table readable; do not squeeze every metric into one tiny IEEE column. Use a separate ten-class recall table or heatmap, a timing/storage table and concise error examples. Every table names the training profile and partition. Display either 0-1 metrics or percentages consistently.

Validate the report, not just your notebook

Check each number pasted into report text and slides against its saved run/stage. Ensure class order, confusion axes and timing units remain correct after formatting. Confirm that rounded values do not imply a different winner from the unrounded selection rule. Use the same run IDs in text, figures and manifests.

Your completion checklist

- Toy calculations and a teammate review confirm metric interpretation and ID alignment. Every final row has complete evidence.
- All ten class recalls and supports are included. The error selection rule and paired comparison groups are documented.
- Inference timing includes HOG when claimed end to end, uses repeated measurements and identifies amortized batch cost.
- The report distinguishes learned weights, stored examples, tree structure and support-vector storage. No model is labeled cost-free.
- You can explain one result, one failure, one comparison limit and the reason final test access was delayed.

Your concrete completion standard

Every reported number can be regenerated from saved predictions or a documented benchmark, and every interpretation stays within what those measurements support.

13 / TEAM DELIVERY

Handoffs and shared file contract

Each person works on their own files, commits meaningful changes, and opens a review request or asks a teammate to review. Share generated evidence with file paths and the exact run ID; a screenshot in a group chat is insufficient.

Sender → receiver	What is handed over; acceptance check
Sudeshhna → everyone	Loader, labels, split_indices.npz, audit.md. Row counts and train/validation IDs agree.
Puneet → Samarth + Neelanshu	Feature functions, HOG settings, feature metadata. A sample ID yields the same vector everywhere.
Puneet + Samarth → Neelanshu	Fitted pipelines, validation predictions, run logs and feature metadata. Every prediction has a split and sample ID.
Neelanshu → Nakul + model owners	Verified tables, error figures, timing method and audit findings. Results trace back to saved predictions.
All four → Nakul	Their report sections, figure captions, references, contribution notes and slides.
Nakul → everyone	Integrated notebook, draft report and slides. Each member checks their section and practices the full demo.

Use these identifiers consistently

sample_id: original row number within the official partition, plus **source_split** = official_train or official_test. Train/validation IDs both refer to official_train. Never use the row number of a shuffled feature cache as the original sample ID.

Prediction CSV: run_id, stage, source_split, sample_id, y_true, y_pred. stage is val or test. Do not invent probabilities when a classifier only exposes decision scores. Preserve deterministic sample order.

Run record: run_id, owner, git_commit, seed, split_file_hash, train_n, val_n, feature_name, feature_dim, feature_config, scaler, estimator, hyperparameters, package_versions, fit_seconds, status, warnings and artifact_paths. Keep validation and test metrics in separate fields/files.

Results CSV: run_id, stage, train_n, feature, model, accuracy, macro_f1, fit_seconds, predict_ms_per_image, end_to_end_ms_per_image, serialized_bytes, complexity_type, complexity_value. Put per-class recall in a separate ten-row table per run. Use seconds, milliseconds and bytes explicitly.

Every handoff includes: what changed, how to run it, expected shapes, the evidence checked, and known limitations. The person receiving it must rerun a small example before accepting it.

14 / TEAM DELIVERY

Individual viva practice

Practice without reading the answers. Explain each idea simply, then open the relevant code or figure as evidence. These are suggested talking points, not answers to memorize word for word.

Everyone must answer these

What is the difference between validation and test?
Validation guides development. Test measures the frozen procedure on held-out examples. Looking at test failures and then tuning would contaminate that final assessment.

Why use both pixels and HOG?
They test whether a fixed description of local edges helps compared with raw intensities. HOG may help some classifiers and hurt others; the experiment must establish the result.

Why is accuracy alone insufficient?
It summarizes total correctness and can hide a weak category. Macro F1 averages class F1 values equally; per-class recall shows which categories are missed.

What can you honestly conclude about a CNN now?
Only that a CNN is a motivated Phase 2 direction. There is no evidence that your future CNN outperforms your ML baseline until that comparison is run.

Neelanshu: questions about your own work

What is a row-normalized confusion diagonal?
Per-class recall, because each true-category row is divided by the number of actual examples of that category.

Does 1 ms/image mean one request takes 1 ms?
Not necessarily. In our proposed benchmark it is amortized batch cost. Single-request latency requires a separately labeled batch-size-one measurement.

Can you choose the best model from test scores?
The selected model was already named using validation. Test rankings can be described, but choosing or tuning from them consumes the test as a selection set.

Five-minute live demonstration

Show your own file and explain one nontrivial choice. Run a small cell or load a saved artifact. Trace one image from its original ID through features to a prediction. Read a validation metric correctly. Explain one failure, one limitation and one improvement reserved for Phase 2. Everyone must also describe the eight-row experiment and the shared split.

15 / TEAM DELIVERY

Suggested 10-day work plan

This is a relative plan, not an official deadline. Budget roughly 18-26 focused hours per person plus unattended compute; beginners may need longer. If you have fewer days, reduce the optional search grid and parallelize member-owned work. Keep data integrity, all eight baseline rows, and honest reporting.

Day	Your focus	Team gate
1	Learn metrics with toy labels	All five understand scope and vocabulary
2	Agree prediction-file schema	Environment works; literature notes started
3	Build evaluator and dummy row	Data/split/feature contracts agreed
4	Check smoke-run predictions	One tiny run reaches the evaluator
5	Audit validation selection rules	Protocol and compute budget fixed
6	Prepare timing/error functions	Eight comparable rows progressing
7	Verify frozen runs; final test	Configurations frozen; test run authorized internally
8	Produce tables and error figures	Verified figures and individual text ready
9	Write evaluation; rehearse viva	One full rehearsal; report compiles
10	Verify report numbers and artifacts	Fresh-run check; contribution log complete

Definition of done for every member

- Your files run in the agreed environment, your inputs and outputs are documented, and a teammate has reviewed one small run.
- Your report section explains what you did, why you did it, what evidence supports it, and what its limits are. Captions and citations are included.
- Your contribution log names real files, commits, experiments and reviews. It records actual work, not an artificial equal number of commits.
- You can explain your own implementation and the shared project during an individual viva. You have practiced a backup demo using saved outputs.

If a run fails, preserve the error and settings. Tell the team early and revise the compute plan consistently. Do not fabricate results, omit failed rows without explanation, or quietly change the split.

16 / TEAM DELIVERY

Sources and how to use them

Course source: CSAI_Project_Instructions_Phase_1_and_Phase_2.md, supplied by you. Its Phase 1 list and individual-viva rule govern the required work. Your pasted Project 11 description supplies the feature/model comparison and metrics. Neither source contains actual results for your team.

Technical references below were checked on 27 October 2026. Stable documentation can change; record the versions actually installed for your experiments. Reference IDs such as [R3] in this guide point to these links. Read and cite original material in your submitted report; this guide is not a substitute for a literature review.

[R1] Fashion-MNIST official repository: data files, label map and official partitions

[R2] Xiao, Rasul and Vollgraf (2017), Fashion-MNIST dataset paper

[R3] scikit-image: HOG API and settings

[R4] scikit-image: HOG worked explanation

[R5] scikit-learn: leakage and preprocessing pitfalls

[R6] scikit-learn: LogisticRegression

[R7] scikit-learn: KNeighborsClassifier

[R8] scikit-learn: RandomForestClassifier

[R9] scikit-learn: SVC

[R10] scikit-learn: model evaluation and metrics

[R11] scikit-learn: stratified train_test_split

[R12] Torchvision: FashionMNIST loader

[R13] Dalal and Triggs (2005): original HOG paper, CVPR

[R14] scikit-learn: LinearSVC alternative and multiclass behavior

[R15] LeCun et al. (1998): author-hosted document-recognition paper record

Suggested research reading beyond API pages

Read Dalal and Triggs (2005), "Histograms of Oriented Gradients for Human Detection" [R13], for the original descriptor. Its task is human detection, so do not transfer its scores to clothing classification. LeCun et al. (1998), "Gradient-Based Learning Applied to Document Recognition" [R15], is useful Phase 2 background; it does not require a Phase 1 CNN implementation.

Citation discipline

For each source record title, authors, year, task, representation, model, evaluation protocol and relevance. Separate a paper's results from your results. Repository leaderboard entries are not automatically controlled comparisons or verified targets. Use actual bibliographic metadata in your LaTeX .bib file; never cite an unread paper as evidence for a specific finding.

Code blocks are starter recipes to adapt and run in your project environment. They have not been run on Fashion-MNIST as part of creating these guides. Toy calculations are illustrative, not measured project performance.